OXVERSE ACADEMY

Backend Development Curriculum

Professional Diploma · Cohort-based · Project-driven

DURATION	TOTAL WEEKS	LEVEL	PROJECTS
3 Months (12 Weeks)	12	Beginner to Industry Ready	15+

CAPSTONE
1 Production-Grade API Platform

PROGRAMME GOAL

To transform beginners into industry-ready backend engineers who can design, build, secure, test, and deploy production-grade APIs and services using modern Python, FastAPI, PostgreSQL, and cloud-native tooling.

OVERVIEW

This 12-week Python-based Backend Development Diploma takes learners from core Python fundamentals through advanced object-oriented and asynchronous programming, into building real-world REST APIs with FastAPI and Pydantic. Learners master relational data modeling with PostgreSQL and SQLAlchemy/Alembic, expand into Redis caching and MongoDB, implement production-grade authentication and OWASP-aware security, process background jobs with Celery, build real-time features with WebSockets, and containerize and deploy applications using Docker and CI/CD pipelines. The program closes with system design, observability, a secondary Django/DRF module, and a capstone in which learners architect, build, secure, test, and deploy a complete production-grade API platform.

No 82, Century Bus Stop, Ago Palace Way, Okota, Lagos · +234 913 869 1147

Course Outline

1. **Week 1:** Python Language Foundations
2. **Week 2:** Object-Oriented & Advanced Python
3. **Week 3:** Async Python & Web Fundamentals
4. **Week 4:** FastAPI Fundamentals
5. **Week 5:** SQL & PostgreSQL Mastery
6. **Week 6:** SQLAlchemy ORM & Database Integration
7. **Week 7:** Authentication, Authorization & Security
8. **Week 8:** Testing, Redis Caching & NoSQL with MongoDB
9. **Week 9:** Background Jobs, Task Queues & Real-Time Communication
10. **Week 10:** Docker, CI/CD & Cloud Deployment
11. **Week 11:** System Design, Observability & Django
12. **Week 12:** Capstone: Production-Grade API Platform

WEEK 1 OF 12

Python Language Foundations

Establish rock-solid Python 3.12+ fundamentals used across every backend system you will ever build.

LEARNING OBJECTIVES

- Set up a professional Python development environment with virtual environments and tooling
- Master Python's core syntax, data types, and control flow
- Write clean, idiomatic Python using PEP 8 conventions
- Understand Python's memory model and variable scoping

DETAILED TOPIC BREAKDOWN

1.1 Environment & Tooling

Installing Python 3.12+ | pyenv for version management | venv vs virtualenv vs poetry | pip and requirements.txt | pyproject.toml structure | VS Code / PyCharm setup | ruff and black formatters | mypy static typing intro | REPL and IPython | Jupyter for exploration | Git basics for Python projects | .gitignore for Python

1.2 Core Syntax & Data Types

Variables and dynamic typing | int, float, complex, bool | Strings and f-strings | String methods and slicing | Numeric operators and precedence | Type casting | None and NoneType | Truthy/falsy values | Multiple assignment and unpacking | Walrus operator := | Comments and docstrings | Input/output basics

1.3 Collections

Lists and list methods | Tuples and immutability | Sets and set operations | Dictionaries and dict methods | Nested data structures | List/dict/set comprehensions | Generator expressions | zip(), enumerate(), map(), filter() | Sorting with key functions | Slicing advanced patterns | Copy vs deepcopy | Named tuples

1.4 Control Flow & Functions

if/elif/else | for and while loops | break, continue, else on loops | match-case statements | Function definitions and return values | Default and keyword arguments | *args and **kwargs | Lambda functions | Scope: local, enclosing, global | Recursion basics | Type hints for functions | Docstring conventions (Google/NumPy style)

1.5 Error Handling & Files

try/except/else/finally | Custom exceptions | Exception chaining | Context managers with 'with' | Reading/writing files | Working with JSON | Working with CSV | pathlib for file paths | Logging module basics | assert statements | Debugging with pdb | Environment variables with os and dotenv

HANDS-ON EXERCISES

- Build a CLI calculator with error handling
- Write comprehension-based data transformers

- Implement a text file word-frequency counter
- Create custom exception hierarchy for a mini app
- Practice unpacking and *args/**kwargs patterns

ASSIGNMENTS

- Build a command-line contact book with file persistence
- Write a script that parses and summarizes a CSV dataset

PROJECTS

- CLI Personal Finance Tracker (file-based storage)

WEEKLY LEARNING OUTCOMES

- Comfortable writing idiomatic Python scripts
- Can debug and handle errors gracefully
- Understands Python data structures deeply
- Able to read/write files and JSON/CSV confidently

WEEKLY ASSESSMENT

Timed coding test covering syntax, data structures, functions, and error handling.

WEEK 2 OF 12

Object-Oriented & Advanced Python

Deepen Python mastery with OOP design, functional patterns, and advanced language features used in production codebases.

LEARNING OBJECTIVES

- Design classes using OOP principles
- Apply advanced Python features: decorators, generators, context managers
- Understand Python's typing system for large codebases
- Structure multi-module Python packages

DETAILED TOPIC BREAKDOWN

2.1 Classes & Objects

class syntax and __init__ | Instance vs class attributes | Instance methods, class methods, static methods | self and cls conventions | Encapsulation and property decorators | __str__ and __repr__ | Operator overloading (__eq__, __add__) | Composition vs inheritance | Multiple inheritance and MRO | super() usage | Abstract base classes (ABC) | dataclasses for boilerplate reduction

2.2 Advanced OOP Patterns

Interfaces via Protocols | Mixins | Class decorators | Metaclasses (intro) | Enum classes | Singleton pattern | Factory pattern | Repository pattern basics | SOLID principles in Python | Design patterns overview | Immutable objects (frozen dataclasses) | Slots for memory optimization (__slots__)

2.3 Functional & Iterators

Iterators and __iter__/__next__ | Generators and yield | yield from and generator delegation | itertools module (chain, groupby, product) | functools (reduce, partial, lru_cache) | Decorators with arguments | Chaining multiple decorators | First-class functions | Closures | Higher-order functions | Memoization patterns | Context managers with contextlib

2.4 Typing & Modules

typing module: List, Dict, Optional, Union | Generics with TypeVar | Protocol typing | TypedDict | Literal types | Structuring packages (__init__.py) | Relative vs absolute imports | Module vs package | Namespace packages | Circular import resolution | Publishing internal packages | mypy strict mode configuration

2.5 Testing Fundamentals

pytest basics and test discovery | Assertions and fixtures | Parametrized tests | Mocking with unittest.mock | Test coverage with coverage.py | conftest.py patterns | Testing exceptions | Setup/teardown patterns | Test organization strategy | TDD workflow intro

HANDS-ON EXERCISES

- Refactor Week 1 CLI app into OOP classes
- Build a generator-based data pipeline
- Write decorators for logging and timing functions
- Implement a custom context manager
- Write pytest suite for a small library

ASSIGNMENTS

- Design a class hierarchy for an e-commerce inventory system
- Build a typed, tested utility library published as an internal package

PROJECTS

- Library Management System (OOP, file-based, fully tested)

WEEKLY LEARNING OUTCOMES

- Can design clean OOP class hierarchies
- Comfortable with decorators, generators, and context managers
- Writes typed, testable Python code
- Understands package structuring best practices

WEEKLY ASSESSMENT

Code review assignment: refactor a procedural script into well-tested OOP code.

WEEK 3 OF 12

Async Python & Web Fundamentals

Understand how the web works and master Python's async model, the backbone of modern high-performance APIs.

LEARNING OBJECTIVES

- Understand HTTP, REST principles, and client-server architecture
- Master Python's asyncio event loop and coroutines
- Differentiate concurrency models: threading, multiprocessing, asyncio
- Build simple async network clients

DETAILED TOPIC BREAKDOWN

3.1 Web & HTTP Fundamentals

HTTP request/response cycle | HTTP methods (GET/POST/PUT/PATCH/DELETE) | Status codes and their meaning | Headers, cookies, sessions | URL structure and query params | REST architectural principles | Richardson maturity model | JSON as data interchange | Content negotiation | CORS explained | API versioning strategies | Idempotency concepts

3.2 Asyncio Core

Event loop concept | async def and coroutines | await keyword | asyncio.run, create_task, gather | Tasks vs coroutines | Cancellation and timeouts | asyncio.sleep vs time.sleep | Async context managers (async with) | Async iterators (async for) | asyncio.Queue | Exception handling in async code | Debugging asyncio apps

3.3 Concurrency Models

GIL explained | Threading module and Lock | Multiprocessing module | ThreadPoolExecutor / ProcessPoolExecutor | When to use async vs threads vs processes | concurrent.futures | CPU-bound vs I/O-bound workloads | Async HTTP clients (httpx) | Async file I/O (aiofiles) | Race conditions and synchronization | Deadlock avoidance | Performance benchmarking

3.4 Networking Basics

sockets module intro | Building a simple TCP echo server | Building a simple HTTP server from scratch | WSGI vs ASGI explained | uvicorn as ASGI server | Request lifecycle in ASGI apps | DNS and networking basics for backend devs | Load balancers conceptually | Reverse proxies (nginx intro) | Environment configuration patterns

HANDS-ON EXERCISES

- Build an async web scraper with httpx and asyncio.gather
- Compare threaded vs async fetch performance
- Build a raw TCP chat server with sockets
- Implement rate-limited async task queue
- Benchmark CPU-bound task across threading/multiprocessing

ASSIGNMENTS

- Build an async multi-URL health checker CLI tool
- Write a report comparing concurrency models with benchmarks

PROJECTS

- Async Web Scraper & Aggregator Tool

WEEKLY LEARNING OUTCOMES

- Understands HTTP/REST deeply
- Comfortable writing asyncio-based programs
- Knows when to choose threads, processes, or async
- Understands ASGI fundamentals before learning FastAPI

WEEKLY ASSESSMENT

Practical exam: build and benchmark an async data-fetching tool.

WEEK 4 OF 12

FastAPI Fundamentals

Start building real APIs with FastAPI, the industry-standard modern Python web framework.

LEARNING OBJECTIVES

- Build RESTful APIs using FastAPI
- Validate data using Pydantic v2 models
- Understand dependency injection in FastAPI
- Generate and use interactive API documentation

DETAILED TOPIC BREAKDOWN

4.1 FastAPI Basics

Project setup and structure | Installing fastapi and uvicorn | Path operations (@app.get, @app.post etc) | Path parameters and type conversion | Query parameters and defaults | Request body parsing | Response models | Status codes with responses | Automatic OpenAPI/Swagger docs | ReDoc documentation | Uvicorn reload and workers | Application factory pattern

4.2 Pydantic v2 Models

BaseModel basics | Field validation and constraints | Field() with defaults and metadata | Custom validators (@field_validator) | Model validators (@model_validator) | Nested models | Optional and Union fields | Enum fields | computed_field | model_config settings | Serialization (model_dump, model_dump_json) | Pydantic Settings for config management

4.3 Routing & Structure

APIRouter for modular routes | Route prefixes and tags | Path operation configuration | Request/response schema separation (DTOs) | Organizing routers/services/schemas folders | Versioned API structure (/api/v1) | Static files serving | Templating with Jinja2 (brief) | Background tasks (BackgroundTasks) | Exception handlers (custom) | HTTPException usage | Global exception handling

4.4 Dependency Injection

Depends() basics | Function-based dependencies | Class-based dependencies | Sub-dependencies | Dependency caching | Dependencies for shared logic (pagination, filters) | Dependency overrides for testing | Path/query dependency reuse | Yield dependencies for cleanup | Global dependencies | Security dependency pattern intro

4.5 Middleware & CORS

Custom middleware | CORSMiddleware configuration | GZip middleware | Request/response logging middleware | Rate limiting middleware concept | Trusted host middleware | Startup/shutdown events (lifespan) | Environment-based configuration | Health check endpoints

HANDS-ON EXERCISES

- Build CRUD endpoints for a Todo API
- Add validation with custom Pydantic validators
- Implement pagination dependency
- Write global exception handlers
- Add request logging middleware

ASSIGNMENTS

- Build a Book Catalog API with full CRUD and validation
- Refactor endpoints into routers/services/schemas structure

PROJECTS

- Task Management REST API (in-memory storage, fully documented)

WEEKLY LEARNING OUTCOMES

- Can scaffold and structure a FastAPI project professionally
- Comfortable with Pydantic validation
- Understands dependency injection deeply
- Can produce clean, documented REST APIs

WEEKLY ASSESSMENT

Build and document a complete CRUD API meeting a given spec.

WEEK 5 OF 12

SQL & PostgreSQL Mastery

Master relational databases and SQL, the foundation of persistent backend data.

LEARNING OBJECTIVES

- Write complex SQL queries confidently
- Design normalized relational schemas
- Understand indexes and query performance
- Use PostgreSQL-specific features effectively

DETAILED TOPIC BREAKDOWN

5.1 SQL Fundamentals

Installing PostgreSQL | psql CLI basics | CREATE TABLE and data types | INSERT, SELECT, UPDATE, DELETE | WHERE clauses and operators | ORDER BY, LIMIT, OFFSET | DISTINCT | Aggregate functions (COUNT, SUM, AVG) | GROUP BY and HAVING | NULL handling | Basic constraints (NOT NULL, UNIQUE, CHECK) | Comments and formatting SQL

5.2 Relationships & Joins

Primary keys and foreign keys | One-to-many relationships | Many-to-many with junction tables | One-to-one relationships | INNER JOIN | LEFT/RIGHT/FULL OUTER JOIN | CROSS JOIN | Self joins | Subqueries | Common Table Expressions (WITH) | UNION and UNION ALL | Set operations (INTERSECT, EXCEPT)

5.3 Schema Design

Normalization (1NF, 2NF, 3NF) | Denormalization trade-offs | ER diagrams | Choosing data types wisely | Indexes: B-tree, GIN, GiST | Composite indexes | Unique constraints and partial indexes | Cascading deletes/updates | Soft deletes pattern | Audit columns (created_at/updated_at) | UUID vs serial primary keys | Schema versioning strategy

5.4 Advanced PostgreSQL

JSONB columns and querying | Full-text search basics | Window functions (ROW_NUMBER, RANK) | Transactions and ACID | Isolation levels | Explain and Explain Analyze | Query optimization techniques | Views and materialized views | Stored procedures/functions (PL/pgSQL intro) | Triggers | Connection pooling concepts (pgbouncer) | Backup and restore basics (pg_dump)

HANDS-ON EXERCISES

- Write complex multi-join reporting queries
- Design a normalized schema for a blog platform
- Practice window functions on sales data
- Optimize a slow query using EXPLAIN ANALYZE
- Write CTEs for hierarchical data

ASSIGNMENTS

- Design and implement a normalized e-commerce database schema
- Write a SQL performance report comparing indexed vs non-indexed queries

PROJECTS

- E-Commerce Database Design with 10+ related tables and analytical queries

WEEKLY LEARNING OUTCOMES

- Writes advanced SQL confidently
- Can design normalized, performant schemas
- Understands indexing and query optimization
- Comfortable with PostgreSQL-specific features

WEEKLY ASSESSMENT

SQL practical exam: schema design plus 10 query challenges.

WEEK 6 OF 12

SQLAlchemy ORM & Database Integration

Integrate PostgreSQL into FastAPI applications using SQLAlchemy 2.0 and manage schema migrations with Alembic.

LEARNING OBJECTIVES

- Model relational data using SQLAlchemy 2.0 ORM
- Perform CRUD operations through the ORM efficiently
- Manage schema evolution using Alembic migrations
- Integrate async database sessions with FastAPI

DETAILED TOPIC BREAKDOWN

6.1 SQLAlchemy Core Concepts

Engine and connection basics | Declarative Base and Mapped classes | Column types and mapping | Table metadata | Sessions and session lifecycle | Sync vs async engine (asyncpg driver) | Connection pooling configuration | Raw SQL execution with text() | Reflection of existing databases

6.2 ORM Modeling

Defining models with Mapped[] and mapped_column | Relationships: relationship() | One-to-many mapping | Many-to-many with association tables | One-to-one mapping | Backref vs back_populates | Cascade options | Lazy loading vs eager loading (joinedload, selectinload) | Hybrid properties | Model mixins for shared columns | Enum columns | Constraints in ORM models

6.3 CRUD & Queries

select() statement construction (2.0 style) | Filtering with where() | Ordering and pagination in ORM | join() in ORM queries | Aggregate queries via ORM | Bulk insert/update/delete | Transactions and commit/rollback | Async session usage (AsyncSession) | Repository pattern with SQLAlchemy | Unit of work pattern | Query performance pitfalls (N+1 problem) | Testing with a transactional rollback pattern

6.4 Alembic Migrations

Installing and initializing Alembic | alembic.ini configuration | Autogenerate migrations | Writing manual migration scripts | Upgrade and downgrade commands | Branching migrations | Data migrations vs schema migrations | Migration naming conventions | Handling migration conflicts in teams | Running migrations in CI/CD | Seeding initial data

6.5 FastAPI + DB Integration

Database session dependency | get_db pattern with yield | Async database dependency | Environment-based DB URLs | Connecting Pydantic schemas with ORM models | from_attributes / orm_mode | Layered architecture: router -> service -> repository -> model | Error handling for DB constraint violations | Testing endpoints with a test database

HANDS-ON EXERCISES

- Model a blog schema with SQLAlchemy relationships
- Write async CRUD repository functions
- Create and run Alembic migrations for schema changes
- Fix an N+1 query problem using eager loading
- Write integration tests against a test DB

ASSIGNMENTS

- Build a full async repository layer for a multi-table domain
- Convert the Task Management API to persist data in PostgreSQL

PROJECTS

- Task Management API v2 (PostgreSQL-backed, migrated with Alembic)

WEEKLY LEARNING OUTCOMES

- Can model complex relational data with SQLAlchemy 2.0
- Manages schema migrations confidently with Alembic
- Understands async DB session handling in FastAPI
- Follows layered architecture best practices

WEEKLY ASSESSMENT

Build a persisted, migrated, tested CRUD module end-to-end.

WEEK 7 OF 12

Authentication, Authorization & Security

Implement secure authentication and authorization systems, and understand core web security principles.

LEARNING OBJECTIVES

- Implement JWT-based authentication in FastAPI
- Understand OAuth2 flows and third-party login
- Apply role-based and permission-based access control
- Recognize and mitigate common web vulnerabilities (OWASP Top 10)

DETAILED TOPIC BREAKDOWN

7.1 Password & Session Security

Password hashing with bcrypt/argon2 | passlib / pwdlib usage | Salting concepts | Storing credentials securely | Session-based vs token-based auth | Secure cookie flags (HttpOnly, Secure, SameSite) | CSRF protection basics | Login/logout flow design | Account lockout and rate limiting | Email verification flow

7.2 JWT & OAuth2

JWT structure (header/payload/signature) | Access tokens vs refresh tokens | Token expiration strategy | OAuth2PasswordBearer in FastAPI | OAuth2 password flow implementation | Refresh token rotation | Revocation/blacklisting strategies | Third-party OAuth2 (Google/GitHub login) | OpenID Connect basics | Storing tokens client-side safely | python-jose / pyjwt libraries

7.3 Authorization

Role-based access control (RBAC) | Permission-based access control | Scopes in OAuth2 | Dependency-based authorization guards | Resource ownership checks | Admin vs user route protection | Multi-tenant authorization concepts | API key authentication for services | Rate limiting per user/role

7.4 OWASP & Security Hardening

OWASP Top 10 overview | SQL injection prevention (parameterized queries) | XSS prevention basics for APIs | CSRF deep dive | Mass assignment vulnerabilities | Insecure direct object references (IDOR) | Security headers (HSTS, CSP, X-Frame-Options) | Secrets management (never commit secrets) | Dependency vulnerability scanning | Input sanitization and validation layers | Logging sensitive data pitfalls | Rate limiting and brute-force protection

HANDS-ON EXERCISES

- Implement JWT login/refresh flow from scratch
- Add RBAC to an existing API
- Implement Google OAuth2 login
- Write tests for auth edge cases (expired/invalid tokens)
- Audit an API for OWASP Top 10 issues

ASSIGNMENTS

- Build a complete auth module: register, login, refresh, logout, RBAC
- Write a security audit report on a provided sample API

PROJECTS

- Secure Auth Service (JWT + OAuth2 + RBAC) reusable across projects

WEEKLY LEARNING OUTCOMES

- Implements production-grade JWT/OAuth2 auth
- Applies RBAC and scopes correctly
- Understands and mitigates OWASP Top 10 risks
- Hardens APIs against common attacks

WEEKLY ASSESSMENT

Security-focused practical: implement auth and pass a vulnerability checklist review.

WEEK 8 OF 12

Testing, Redis Caching & NoSQL with MongoDB

Ensure code quality with rigorous testing, and expand data layer skills with caching and document databases.

LEARNING OBJECTIVES

- Write comprehensive automated test suites with pytest
- Implement caching strategies using Redis
- Model and query data in MongoDB
- Decide when to use SQL vs NoSQL vs caching layers

DETAILED TOPIC BREAKDOWN

8.1 Testing FastAPI Apps

TestClient / httpx AsyncClient | Testing endpoints end-to-end | Fixtures for test DB setup/teardown | Factory pattern for test data (factory_boy) | Mocking external services | Dependency overrides for tests | Testing auth-protected routes | Parametrized endpoint tests | Coverage reporting and thresholds | Test pyramid: unit/integration/e2e | Continuous testing workflow

8.2 Redis Fundamentals

Installing and running Redis | Redis data types: strings, hashes, lists, sets, sorted sets | redis-py / redis.asyncio client | Basic caching pattern (cache-aside) | TTL and expiration strategies | Cache invalidation strategies | Rate limiting with Redis | Session storage in Redis | Pub/Sub messaging basics | Redis as a message broker for Celery | Distributed locks with Redis

8.3 MongoDB Fundamentals

Document model concept | Installing MongoDB / Atlas setup | Collections and documents | CRUD with pymongo/motor (async) | Query operators ($gt, $in, $regex etc) | Indexing in MongoDB | Aggregation pipeline basics ($match, $group, $project) | Embedding vs referencing documents | Schema validation in MongoDB | ODMs: Beanie / MongoEngine overview | When to choose MongoDB over PostgreSQL

8.4 Polyglot Persistence Strategy

SQL vs NoSQL trade-offs | Caching layer placement in architecture | CAP theorem basics | Combining PostgreSQL + Redis + MongoDB in one system | Data consistency considerations | Read-heavy vs write-heavy optimization | Denormalization for read performance | Cache warming strategies

HANDS-ON EXERCISES

- Write full pytest coverage for auth service
- Implement Redis caching for a slow database query
- Build a MongoDB-backed analytics logging feature
- Implement Redis-based rate limiter middleware
- Write aggregation pipeline for reporting

ASSIGNMENTS

- Add a caching layer to the Task Management API with measured performance improvement
- Build a MongoDB-based activity log/audit trail service

PROJECTS

- Analytics & Activity Log Service (MongoDB + Redis caching)

WEEKLY LEARNING OUTCOMES

- Writes high-coverage automated test suites
- Implements effective caching strategies with Redis
- Comfortable modeling and querying MongoDB
- Can architect polyglot persistence solutions

WEEKLY ASSESSMENT

Deliver a tested feature with measurable caching performance gains.

WEEK 9 OF 12

Background Jobs, Task Queues & Real-Time Communication

Handle asynchronous workloads with Celery and build real-time features using WebSockets.

LEARNING OBJECTIVES

- Implement asynchronous background task processing with Celery
- Schedule periodic tasks reliably
- Build real-time features using WebSockets in FastAPI
- Design event-driven backend components

DETAILED TOPIC BREAKDOWN

9.1 Celery Fundamentals

Celery architecture (broker, worker, backend) | Installing Celery with Redis/RabbitMQ broker | Defining tasks with @app.task | Calling tasks with .delay() and .apply_async() | Task result backends | Retries and error handling in tasks | Task chaining and chords | Task routing and queues | Celery worker concurrency models | Monitoring with Flower

9.2 Scheduling & Reliability

Celery Beat for periodic tasks | Cron-style scheduling | Idempotent task design | Task deduplication strategies | Dead-letter handling for failed tasks | Task timeouts | Rate limiting tasks | Graceful worker shutdown | Scaling workers horizontally | Task observability and logging

9.3 WebSockets in FastAPI

WebSocket protocol basics | @app.websocket route handler | Accepting and closing connections | Broadcasting to multiple clients | Connection manager pattern | Authentication for WebSocket connections | Handling disconnects gracefully | WebSocket vs Server-Sent Events (SSE) | Scaling WebSockets across multiple instances (Redis pub/sub) | Testing WebSocket endpoints

9.4 Event-Driven Patterns

Producer/consumer pattern | Message queues overview (RabbitMQ vs Redis vs Kafka intro) | Event sourcing concept overview | Webhooks: sending and receiving | Retry with exponential backoff | Outbox pattern for reliability | Designing decoupled services | Async email/notification sending patterns

HANDS-ON EXERCISES

- Build a Celery task to send emails asynchronously
- Schedule a periodic data cleanup task with Celery Beat
- Build a WebSocket-based live chat feature
- Implement webhook receiver with signature verification
- Add retry logic with exponential backoff to a task

ASSIGNMENTS

- Build an async notification system (email + in-app) using Celery
- Build a real-time order-status WebSocket feature for an e-commerce API

PROJECTS

- Real-Time Notification & Chat Backend (Celery + WebSockets + Redis)

WEEKLY LEARNING OUTCOMES

- Implements reliable background job processing with Celery
- Schedules and monitors periodic tasks
- Builds real-time WebSocket features confidently
- Understands event-driven backend architecture

WEEKLY ASSESSMENT

Deliver a working background-job and real-time feature with monitoring in place.

WEEK 10 OF 12

Docker, CI/CD & Cloud Deployment

Package, automate, and deploy Python backend applications like a professional engineering team.

LEARNING OBJECTIVES

- Containerize FastAPI applications with Docker
- Orchestrate multi-service apps with Docker Compose
- Build CI/CD pipelines for automated testing and deployment
- Deploy applications to cloud infrastructure

DETAILED TOPIC BREAKDOWN

10.1 Docker Fundamentals

Docker concepts: images, containers, layers | Writing a Dockerfile for FastAPI | Multi-stage builds for smaller images | .dockerignore best practices | Environment variables in containers | Docker networking basics | Volumes for persistent data | Docker Compose for multi-service apps (app + db + redis) | Health checks in Docker Compose | Debugging containers (logs, exec)

10.2 CI/CD Pipelines

CI/CD concepts overview | GitHub Actions workflow syntax | Running pytest in CI | Linting/formatting checks in CI (ruff, black) | Building and pushing Docker images in CI | Automated database migration step in pipeline | Environment secrets management in CI | Branch protection and PR checks | Semantic versioning and release tagging | Deployment approval gates

10.3 Cloud Deployment

Deployment targets overview (Render, Railway, AWS, GCP, Azure) | Deploying with Docker to a VPS | Reverse proxy setup with nginx | Setting up gunicorn+uvicorn workers for production | Environment configuration for production | Managed PostgreSQL services | Managed Redis services | Domain and SSL setup (Let's Encrypt/Certbot) | Zero-downtime deployment strategies | Rollback strategies

10.4 Infrastructure Basics

Introduction to AWS core services (EC2, RDS, S3, ECS) | Containers on AWS ECS/Fargate overview | Environment variable/secret management in cloud (AWS Secrets Manager) | Basic Infrastructure as Code concept (Terraform overview) | Load balancing basics in cloud | Autoscaling concepts | CDN basics for static assets | Cost-awareness for cloud resources

HANDS-ON EXERCISES

- Dockerize the Task Management API with Compose (app+db+redis)
- Write a GitHub Actions pipeline running tests and lint
- Deploy a containerized API to a cloud VPS
- Configure nginx as reverse proxy with SSL
- Set up a rollback-capable deployment workflow

ASSIGNMENTS

- Fully containerize and deploy a multi-service backend application
- Write a CI/CD pipeline that tests, builds, and deploys on merge to main

PROJECTS

- Production Deployment Pipeline for the Task Management Platform

WEEKLY LEARNING OUTCOMES

- Containerizes multi-service backend apps confidently
- Builds functioning CI/CD pipelines
- Deploys and manages APIs on cloud infrastructure
- Understands production infrastructure basics

WEEKLY ASSESSMENT

Demonstrate a live deployed API reachable via CI/CD-triggered pipeline.

WEEK 11 OF 12

System Design, Observability & Django

Think like a backend architect: design scalable systems, add observability, and learn Django as a secondary framework for full-featured apps.

LEARNING OBJECTIVES

- Apply system design principles to backend architecture
- Implement logging, monitoring, and tracing for production systems
- Understand Django's MTV architecture and ORM as an alternative to FastAPI
- Design scalable, fault-tolerant backend systems

DETAILED TOPIC BREAKDOWN

11.1 System Design Fundamentals

Client-server and n-tier architecture | Vertical vs horizontal scaling | Load balancing strategies | Caching layers in system design | Database sharding and replication | CAP theorem revisited | Message queues in system design | Rate limiting algorithms (token bucket, sliding window) | API Gateway pattern | Microservices vs monolith trade-offs | Designing a URL shortener (case study) | Designing a notification system (case study)

11.2 Observability

Structured logging with structlog/loguru | Log levels and log aggregation (ELK/Loki overview) | Application metrics with Prometheus | Grafana dashboards overview | Distributed tracing with OpenTelemetry | Health check and readiness/liveness probes | Error tracking with Sentry | Alerting basics | Performance profiling (py-spy, cProfile) | APM tools overview

11.3 Django Fundamentals

Django project vs app structure | Django MTV pattern (models/templates/views) | Django ORM models and migrations | Django admin panel | Django URLs and views (function-based & class-based) | Django REST Framework (DRF) serializers | DRF viewsets and routers | DRF authentication/permissions | Django settings and environments | When to choose Django over FastAPI

11.4 API Design Best Practices

RESTful resource naming conventions | HATEOAS overview | API pagination strategies | Filtering and sorting query patterns | Error response standardization (RFC 7807) | API versioning strategies revisited | Rate limiting exposure to clients | OpenAPI spec customization | GraphQL overview and comparison to REST | gRPC overview for internal services

HANDS-ON EXERCISES

- Design a scalable system architecture diagram for a ride-sharing app
- Add structured logging and Sentry integration to an API
- Build a small CRUD app in Django REST Framework
- Implement standardized error responses across an API

- Add Prometheus metrics endpoint to a FastAPI app

ASSIGNMENTS

- Write a system design document for a scalable social media backend
- Rebuild one module of the Task Management API using Django REST Framework

PROJECTS

- System Design Case Study + Django REST Framework Mini-Service

WEEKLY LEARNING OUTCOMES

- Can reason about and design scalable backend systems
- Implements observability tooling in production apps
- Understands Django/DRF as an alternative backend framework
- Applies API design best practices consistently

WEEKLY ASSESSMENT

Present a system design document and a working observability-instrumented service.

WEEK 12 OF 12

Capstone: Production-Grade API Platform

Integrate everything learned into one production-grade, deployed, tested, secure, and documented API platform.

LEARNING OBJECTIVES

- Architect and build a full-featured production backend platform end-to-end
- Apply security, testing, caching, and background processing cohesively
- Deploy the platform with a full CI/CD pipeline and observability
- Present and defend architectural decisions like a professional engineer

DETAILED TOPIC BREAKDOWN

12.1 Planning & Architecture

Requirements gathering and scoping | Domain modeling and ER diagram design | Choosing the tech stack and justifying decisions | API contract design (OpenAPI-first) | Defining service boundaries | Project folder architecture (routers/services/repositories/schemas) | Task breakdown and milestone planning | Risk assessment for the build

12.2 Core Implementation

Implementing authentication and RBAC | Implementing core domain CRUD modules | Integrating PostgreSQL via SQLAlchemy + Alembic | Integrating Redis caching layer | Integrating Celery background jobs | Adding WebSocket real-time feature | Implementing pagination, filtering, sorting | Writing comprehensive Pydantic validation

12.3 Quality & Hardening

Writing unit and integration test suites | Achieving meaningful test coverage | Running security review against OWASP Top 10 | Load testing with locust/k6 | Optimizing slow queries | Adding structured logging and error tracking | Writing OpenAPI documentation polish | Handling edge cases and input validation gaps

12.4 Deployment & Delivery

Dockerizing the full platform | Setting up CI/CD pipeline (test, build, deploy) | Deploying to a cloud environment | Setting up monitoring dashboards | Writing a professional README and architecture doc | Preparing a live demo | Preparing an architecture defense presentation | Post-launch monitoring plan

HANDS-ON EXERCISES

- Conduct a peer code review of a classmate's capstone module
- Run a load test and optimize based on results
- Write an incident response runbook for the platform
- Perform a self-audit against the OWASP checklist

ASSIGNMENTS

- Submit complete, deployed capstone project with documentation
- Deliver a live architecture defense presentation to instructors/peers

PROJECTS

- Capstone: Full Production-Grade API Platform (auth, caching, background jobs, real-time, deployed with CI/CD)

WEEKLY LEARNING OUTCOMES

- Delivers a fully functional, deployed, production-grade backend platform
- Demonstrates mastery of the complete backend engineering lifecycle
- Can defend architectural and technical decisions confidently
- Is portfolio-ready and interview-ready for backend engineering roles

WEEKLY ASSESSMENT

Final capstone evaluation: live demo, code review, and architecture defense.